

A Comprehensive Guide to Measuring Software Energy Consumption in Linux

Part 1: The Foundation of Software Energy Measurement

Section 1: The Silent Cost of Computation: Why We Must Measure

The digital infrastructure that underpins modern society, from cloud computing and artificial intelligence to the billions of connected devices in our pockets, operates on a foundation of electrical energy. While software itself is intangible, its execution has a profound and rapidly growing physical consequence: the consumption of vast amounts of power. Understanding and measuring this consumption is no longer a niche academic pursuit but a critical discipline for responsible and efficient software engineering. This section establishes the context for this necessity, tracing the path from global energy trends down to the individual lines of code that drive them.

The Escalating Energy Footprint of the Digital World

The Information and Communication Technology (ICT) sector has become one of the world's most significant energy consumers. In 2019, software systems alone were estimated to be responsible for 6% of global energy use, a figure that surpassed the gas emissions of the entire civil aviation industry.¹ Projections indicate a staggering trajectory; by the end of this decade, the ICT sector could consume as much as 20% of all electricity generated worldwide.² This trend is not sustainable without a concerted effort to improve efficiency at every level of the technology stack.

At the heart of this escalating demand are data centers, the massive, centralized facilities that power cloud services, data storage, and increasingly, artificial intelligence workloads. These facilities are exceptionally energy-intensive, consuming between 5 and 10 times more energy per square foot than a typical office building.³ The global electricity demand from data centers is projected to double between 2022 and 2026, an unprecedented rate of growth fueled largely by the widespread adoption of AI and the continued migration of services to the cloud.⁴ A single large "hyperscale" data center can have a power demand of 100 megawatts (MW) or more, equivalent to the annual electricity consumption of hundreds of thousands of electric cars.⁷

This immense and concentrated energy demand places a significant strain on public infrastructure. In several regions, clusters of new data centers are disrupting the delicate balance of supply and demand on local power grids, leading to higher wholesale electricity prices and increased demand-related charges that are passed on to all consumers, including residential households.³ In Ireland, data centers now account for over 20% of the country's total electricity consumption, while in some US states, that figure has surpassed 10%.⁷ This strain not only drives up costs but also increases the risk of power outages, as utility providers struggle to upgrade infrastructure at the same pace as data center construction.³

Why Software Efficiency Matters

The connection between the global energy crisis and the daily work of a software developer may not seem immediately obvious, but it is direct and undeniable. Every architectural choice, every algorithm implemented, and every line of code written translates into a series of physical operations on hardware, and these operations consume electricity.⁸ Recognizing this causal chain is the first step toward building more sustainable software.

Linking Code to Carbon

Software directs the activity of hardware components like CPUs, memory, and network interfaces.¹⁰ The electricity required to power these components is drawn from a grid that, in many parts of the world, is still heavily reliant on fossil fuels. As of 2023, approximately 56% of the electricity used to power data centers in the United States came from fossil fuel sources.¹¹ Consequently, inefficient software—code that performs unnecessary computations, makes redundant data transfers, or keeps hardware in high-power states for longer than necessary—directly contributes to an increase in greenhouse gas (GHG) emissions.² The act

of writing and running software is an act with a real-world carbon footprint.

Operational Costs and Performance

Beyond the environmental impact, energy consumption is a major operational expenditure for any organization running software at scale.¹³ For data centers, energy is often one of the largest components of the total cost of ownership. Therefore, measuring and optimizing the energy consumption of applications leads to direct and significant cost savings.⁸ This financial incentive is a powerful driver for the adoption of energy-aware development practices.

Furthermore, a focus on energy efficiency often yields substantial performance improvements. Energy-efficient code is, by its nature, code that minimizes wasteful operations. This typically results in software that runs faster, is more responsive, and provides a better user experience.⁸ Optimizing for energy forces developers to scrutinize their code for inefficiencies, which benefits performance and reliability as a side effect.

Compliance and Corporate Responsibility

The growing awareness of the technology sector's environmental impact has led to increased regulatory and market pressure. A new set of guidelines and regulations, broadly categorized under Environmental, Social, and Governance (ESG) frameworks, are compelling companies to disclose their sustainability efforts and report on their carbon emissions.¹⁴ Measuring the energy consumption of software is becoming a fundamental requirement for accurate GHG emissions reporting and for demonstrating a credible commitment to sustainability.¹⁷

This shift transforms the practice of energy measurement from a technical optimization into a core business imperative. The motivation for a developer to master these techniques is therefore multifaceted. It begins with environmental stewardship but extends directly to the company's financial health through cost reduction, the quality of the product through performance improvements, and the company's public standing and regulatory compliance through ESG reporting. In an increasingly energy-conscious world, the ability to measure and optimize the energy footprint of software is a critical skill for the modern engineer.

Section 2: Core Principles of Energy Measurement

Before diving into the specific tools and techniques for measuring software energy consumption, it is essential to establish a firm understanding of the fundamental principles involved. This includes clarifying the physics of power and energy, understanding the relationship between software and hardware, and recognizing the two primary methodologies for measurement. These core concepts provide the necessary foundation for interpreting results correctly and making informed decisions about which measurement strategy to employ.

Distinguishing Power (Watts) from Energy (Joules)

In the context of computing, the terms "power" and "energy" are often used interchangeably, but they represent distinct physical quantities. A precise understanding of this distinction is crucial for accurate measurement and analysis.

- **Power**, measured in **Watts (W)**, is the *instantaneous rate* at which energy is being consumed or transferred. It is a snapshot of how much electricity a component, such as a CPU, is drawing at a specific moment in time.¹⁸ A processor might draw 20 W while idle and spike to 100 W under a heavy computational load. Power tells you *how fast* energy is being used.
- **Energy**, measured in **Joules (J)**, is the total amount of work done or heat generated over a period. It is the integral of power over time; for a constant power draw, $\text{Energy} = \text{Power} \times \text{Time}$.¹⁸ The "energy consumption" of a program is the total number of Joules it causes the hardware to use from its start to its finish. A short, high-power task might consume less total energy than a long, low-power task. Energy tells you *how much* work was done.

When measuring a program, the goal is almost always to determine its total **energy** consumption. However, most measurement tools, whether hardware or software, sample the instantaneous **power** draw at regular intervals. To calculate the total energy, these power readings are then integrated over the program's execution time.

The Software-Hardware Interaction

A foundational concept in this field is that software, in and of itself, does not directly consume energy. Software is merely information—a set of instructions stored on a physical medium.²⁰ It is the physical execution of these instructions by hardware components that consumes

electrical energy.

When a program runs, its instructions direct the activity of the Central Processing Unit (CPU), Dynamic Random-Access Memory (DRAM), storage devices, and network interfaces.¹⁰ Each of these hardware components has an energy cost associated with its operation. For example, fetching an instruction from memory consumes energy, executing an arithmetic operation in the CPU's ALU consumes energy, and transmitting a data packet over a network consumes energy.¹⁰

Therefore, measuring the energy consumption of a software program is fundamentally an *indirect* measurement. We are not measuring a property of the software itself, but rather the energy consumed by the hardware *as a consequence* of executing that software's instructions.²⁰ This distinction is critical because it means that a program's energy footprint is not an intrinsic property but is dependent on the specific hardware and system environment in which it is run.

The Methodological Spectrum: Direct vs. Indirect Measurement

There are two primary approaches to quantifying the energy consumed during a program's execution: direct measurement using external hardware and indirect measurement (or estimation) using software-based tools. These two methodologies exist on a spectrum, representing a fundamental trade-off between accuracy and accessibility.

Direct Measurement (Hardware Power Meters)

Direct measurement is considered the "gold standard" for accuracy in energy analysis. This method involves using external physical instruments to measure the actual electrical current and voltage flowing into a computing device or a specific component.

- **Definition:** This approach utilizes devices such as wall-socket power meters, metered Power Distribution Units (PDUs) in data centers, or high-precision benchtop power supplies connected directly to a component's power rails.²¹
- **Advantages:**
 - **High Accuracy:** Because they measure the actual flow of electricity, hardware meters provide the most accurate and reliable data, serving as the ground truth against which software-based methods are often validated.²⁵
 - **Non-Invasive:** This method does not add any software overhead to the system under test. The measurement process happens externally and does not interfere with

the program's execution, which could otherwise alter the results.²⁰

- **Disadvantages:**

- **Cost and Complexity:** High-precision measurement hardware can be expensive and requires physical access to the system, making it difficult and costly to deploy at scale.²²
- **Coarse Granularity:** Most external meters measure the total power consumption of an entire system (e.g., everything plugged into a wall socket). They cannot easily attribute that consumption to a specific hardware component (like the CPU vs. the GPU), let alone a single software process or function running on that hardware.²⁰

Indirect Measurement (Software-Based Estimation)

Indirect measurement is the most common and accessible method for developers and is the primary focus of this guide. This approach uses software to read data from internal hardware sensors and performance counters and then applies a mathematical model to *estimate* the energy consumption.

- **Definition:** This method leverages built-in hardware features, like Intel's Running Average Power Limit (RAPL), which expose energy-related data to the operating system. Software tools then read this data and present it to the user.²⁰
- **Advantages:**
 - **Accessibility and Scalability:** It requires no additional hardware and can be easily deployed across many machines, making it highly scalable and cost-effective.²²
 - **Fine Granularity:** Software-based methods can often provide energy data for specific hardware domains (e.g., CPU package, DRAM) and, through further modeling, can be used to attribute energy consumption to individual processes or containers.²⁸
- **Disadvantages:**
 - **Estimation, Not Measurement:** It is crucial to remember that this is an estimation. Its accuracy is dependent on the quality of the underlying hardware sensors and the power models they use, which are often proprietary and not fully documented by the hardware vendor.²⁰ Studies have shown that these estimations can sometimes deviate significantly from direct hardware measurements.²⁵
 - **Invasiveness:** The act of running the measurement software itself consumes system resources (CPU, memory), creating an "observer effect" where the measurement process adds overhead that can slightly alter the energy consumption of the system being measured.²⁰

The entire field of practical software energy analysis for developers is built upon the decision to accept the potential inaccuracies of indirect estimation in exchange for the invaluable

benefits of accessibility, scalability, and granularity. The challenge is not simply to find a tool that produces a number, but to understand the context, limitations, and trade-offs of that tool. This allows a practitioner to know when a measurement is "good enough" for a comparative analysis (e.g., A/B testing a code change) versus when a higher degree of absolute accuracy might be required. This foundational understanding frames the subsequent exploration of specific Linux tools and their underlying technologies.

Part 2: The Linux Energy Measurement Stack

To effectively measure software energy consumption in Linux, it is essential to understand the layers of technology that make this possible. The process begins deep within the CPU hardware, is exposed to the operating system through specific interfaces, and is finally made accessible to user-space tools through a standardized kernel framework. This part of the report dissects this stack, starting with the foundational hardware feature—Intel's Running Average Power Limit (RAPL)—and moving up to the Linux kernel's powercap framework, which provides the gateway for all modern measurement tools.

Section 3: The Heart of the Matter: Intel RAPL

At the core of modern software-based energy measurement on x86 platforms lies Intel's Running Average Power Limit (RAPL) technology. It is a hardware feature integrated into the processor that provides a low-level interface for both monitoring energy consumption and enforcing power limits.

Architecture and History of Running Average Power Limit (RAPL)

RAPL was first introduced in 2011 with the "Sandy Bridge" family of microprocessors. Its primary purpose was to enable power capping, a mechanism for ensuring that a processor operates within a specified thermal and power budget. It achieves this by allowing software to set power limits, which the hardware then enforces by dynamically adjusting the processor's voltage and frequency, a technique known as Dynamic Voltage and Frequency Scaling (DVFS).²⁹

A critical secondary function of RAPL, and the one most relevant to this discussion, is its ability to report energy consumption. The processor maintains a set of internal counters that track the energy consumed by different parts of the chip. This information is exposed to the operating system through a series of Model-Specific Registers (MSRs), which are special-purpose control registers in the x86 architecture.²⁹ These energy status MSRs are updated at a very high frequency—typically around every 1 millisecond (1 kHz), and even faster on more recent architectures—which allows for fine-grained and near-real-time monitoring of energy usage.²⁶

Dissecting the RAPL Domains

RAPL does not provide a single, monolithic energy value for the entire processor. Instead, it breaks down the measurement into several distinct "domains," allowing for a more granular analysis of where energy is being consumed. The availability of these domains can vary depending on the specific processor model (e.g., client vs. server), but the most common ones include ³¹:

- **Package (pkg):** This is the most comprehensive domain, representing the energy consumed by the entire CPU socket. It includes the sum of all processor cores as well as the "uncore" components, such as the Last-Level Cache (L3), integrated graphics, and memory controller. For most system-level measurements, the pkg domain is the most important value.
- **PP0 (Power Plane 0):** This domain measures the energy consumed exclusively by all the processor cores on the socket. It is a sub-domain of the package.
- **PP1 (Power Plane 1):** This domain measures the energy consumed by the integrated GPU. It is primarily found on client processors and is often absent on server-grade CPUs that do not have integrated graphics.³¹
- **DRAM:** This domain reports the energy consumed by the Dynamic Random-Access Memory (DRAM) modules that are attached to the processor's integrated memory controller. This allows for the analysis of memory-intensive workloads.
- **PSys (Platform):** Introduced in more recent architectures like Skylake, the PSys domain provides a more holistic measurement of the entire System-on-a-Chip (SoC), which can include the CPU package, memory, and other devices powered directly by the processor's integrated power delivery mechanism. When available, it offers an even more complete picture than the pkg domain.³¹

By reading the energy counters from these different domains, a developer can gain insights into whether a program's energy footprint is dominated by CPU computation (high PPO consumption), memory access (high DRAM consumption), or a combination of factors.

A Critical Review of RAPL Accuracy

The reliability of RAPL data is a subject of considerable academic research and a critical point of understanding for any practitioner. The term "accurate" is often used to describe RAPL, but its meaning is highly context-dependent.

A key factor in RAPL's evolution is the distinction between its implementation in early versus later processor generations. In its initial form on Sandy Bridge, RAPL was primarily a *software power model* that used hardware performance counters (e.g., cache misses, instructions retired) to *estimate* energy consumption.²² However, starting with the "Haswell" microarchitecture, Intel integrated fully digital voltage regulators (FIVR) onto the chip. This allowed RAPL to transition from a pure estimation model to a system based on

actual power measurement, significantly improving its fidelity.³⁴ This historical detail is crucial; measurements on a post-Haswell CPU are fundamentally more direct and reliable than those on an older Sandy Bridge system.

Despite these improvements, numerous studies comparing RAPL's output to measurements from high-precision external power meters have revealed discrepancies. The nature of these discrepancies is key to understanding RAPL's utility:

1. **Systematic Offset:** Several studies have found that while RAPL's reported power values are lower than the true physical measurements, the *trends and patterns* of power consumption are highly correlated. In other words, RAPL correctly identifies when power usage increases or decreases, but the absolute value it reports is consistently offset by a certain amount.²⁴ This suggests that RAPL is highly effective for *comparative analysis*. For example, if a code optimization reduces the RAPL-reported energy consumption by 20%, it is very likely that the true physical energy consumption also decreased by a similar proportion. The systematic offset is effectively cancelled out in such A/B comparisons.
2. **Variable Error:** Other studies have reported more significant and variable error margins. One analysis of a matrix multiplication workload found that RAPL's reported power had an error of up to 22% compared to physical measurements.²⁵ The accuracy can also be influenced by the specific workload and by processor features like Hyper-Threading, which can sometimes confuse the power model.²⁶

The synthesis of these findings leads to a practical, nuanced conclusion. For the purpose of *absolute energy reporting*—for instance, to claim that a specific process consumed exactly 15.2 Joules for a compliance report—RAPL data should be treated with caution. The reported value is a high-quality estimation but may not be the ground truth. However, for the far more common engineering task of *relative optimization*—determining whether change A is more

energy-efficient than change B—RAPL is an exceptionally powerful and reliable tool. Its high frequency and domain-specific granularity provide invaluable insights for developers seeking to improve their code. A practitioner should first identify their CPU's microarchitecture to understand the underlying measurement technology and then clearly define their goal (comparative vs. absolute) to correctly interpret the results.

Section 4: The Kernel's Gateway: The powercap Framework

While Intel RAPL provides the raw hardware capability for energy measurement, direct access to its Model-Specific Registers (MSRs) is complex and requires privileged, kernel-level permissions. To make this powerful feature accessible to user-space applications in a safe, standardized, and convenient way, the Linux kernel developers created the Power Capping Framework, commonly known as powercap.

Purpose and Design

The powercap framework is a kernel subsystem that abstracts underlying power management technologies, like Intel RAPL, and exposes them to user space through the sysfs virtual file system.³⁶ This design adheres to the classic Unix philosophy of "everything is a file." Instead of requiring applications to use special system calls or have direct hardware access,

powercap presents the energy counters and power limits of each RAPL domain as simple text files that can be read from (for monitoring) or written to (for capping).

This abstraction layer is profoundly important. It democratizes access to RAPL, allowing unprivileged users to read energy data and enabling the development of a wide range of user-space tools without each tool needing to implement its own complex, kernel-specific MSR-handling code. The powercap framework is the standardized bridge between the hardware and the software tools discussed in the next part of this guide.

Navigating the `/sys/class/powercap/` Interface

The powercap framework's interface is located in the `/sys/class/powercap/` directory. It is

organized into a logical tree structure that mirrors the physical hierarchy of the hardware.³⁶

A typical powercap hierarchy for a system with an Intel RAPL-enabled processor looks like this:

```
/sys/class/powercap/
└─ intel-rapl/
   └─ intel-rapl:0/
      ├── constraint_0_name
      ├── constraint_0_power_limit_uw
      ├── constraint_0_time_window_us
      ├── energy_uj
      ├── max_energy_range_uj
      ├── name
      └─ intel-rapl:0:0/
         ├── energy_uj
         ├── name
         └─ ...
   └─ intel-rapl:1/
      └─ ...
```

The key components of this hierarchy are:

- **Control Type:** The top-level directory, named after the underlying technology (e.g., intel-rapl). This allows the framework to support other technologies in the future.³⁷
- **Zone:** A directory representing a specific RAPL domain that can be controlled and monitored. For example, intel-rapl:0 typically corresponds to the package domain of the first CPU socket (CPU 0). The name file within this directory will confirm this (e.g., containing the string "package-0").³⁶
- **Subzone:** A nested directory representing a sub-domain. For example, intel-rapl:0:0 might be the core domain (PP0) within package-0, while intel-rapl:0:1 could be the uncore domain (PP1).³⁶

Within each zone and subzone directory, several key files provide the interface for measurement and control:

- **energy_uj:** This is the most important file for measurement. It is a read-only file containing a 64-bit integer that represents the total accumulated energy consumed by that zone, in **micro-Joules (μJ)**, since the system was booted or the counter was last reset.³⁷

- **max_energy_range_uj**: A read-only file indicating the maximum value the energy_uj counter can reach before it overflows and wraps around to zero. This value is typically very large, but for long-running measurements, it is a factor to be aware of.³⁷
- **name**: A read-only file containing a human-readable string that identifies the zone, such as package-0, core, uncore, or dram.³⁶
- **constraint_*_power_limit_uw** and **constraint_*_time_window_us**: These are readable and writable files used to set power caps in micro-Watts (μW) over a specific time window in micro-seconds (μs). While not used for measurement, they are an integral part of the framework's control capabilities.³⁷

Directly Reading Energy Counters

The elegance of the powercap framework is that the fundamental act of energy measurement can be performed with a simple shell command. To measure the energy consumed by the entire first CPU package, one can read the energy_uj file:

Bash

```
$ cat /sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj
1357924680
```

To measure the energy consumption of a specific program, the process is straightforward:

1. Read the value of energy_uj and store it.
start_energy=\$(cat /sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj)
2. Execute the program to be measured.
./my_program
3. Read the value of energy_uj again.
end_energy=\$(cat /sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj)
4. Calculate the difference.
consumed_uj=\$((end_energy - start_energy))
consumed_j=\$(echo "\$consumed_uj / 1000000" | bc -l)

This simple sequence of commands encapsulates the core principle of software-based energy measurement in Linux. All the higher-level tools discussed in the next section are, at their core, sophisticated wrappers around this fundamental process of reading and differencing the values in the powercap sysfs interface. Understanding this foundation is key

to troubleshooting and to appreciating the value that these tools add in terms of automation, analysis, and presentation.

Part 3: A Practitioner's Toolkit for Energy Profiling

With a solid understanding of the underlying hardware and kernel interfaces, a practitioner can now turn to the user-space tools that make energy measurement practical and efficient. The Linux ecosystem offers a variety of command-line utilities, each with a different design philosophy and primary use case. This section provides a detailed, hands-on guide to the most important of these tools, moving from the general-purpose profiler `perf` to more specialized diagnostic and monitoring utilities. A comparative overview helps in selecting the right tool for a specific task.

Comparison of Linux Energy Measurement Tools

Choosing the appropriate tool depends on the measurement goal. Is the objective to get a single energy number for a program run, to diagnose system-wide inefficiencies, to monitor power trends over time, or to profile on a specific vendor's hardware? The following table provides an at-a-glance comparison to guide this decision.

Tool Name	Primary Use Case	Measurement Granularity	Interface	Key Command Example	Strengths	Limitations
perf	General-purpose performance & energy profiling of a command or	Per-command, System-wide. Reports on hardware domains (pkg,	Command-line.	<code>perf stat -e power/energy-pkg/ ./my_app</code>	Integrated into the kernel, low overhead, versatile for many types of profiling	Output is a simple aggregate; not ideal for time-series analysis or

	system.	core, etc.).			beyond energy. ⁴⁰	diagnosin g system-w ide idle issues.
powertop	Diagnosi ng sources of system wake-ups and tuning for power efficiency .	System- wide, with per-proc ess wake-up attributio n. Power estimatio n per device/pr ocess.	Interactiv e TUI, HTML/CS V reports.	sudo powertop --auto-tu ne	Excellent for identifyin g "power bugs" and applying optimizati ons. Provides detailed C-state/P -state stats. ⁴²	Power estimates require calibratio n and can be less precise. Primarily a diagnosti c, not a program matic measure ment tool.
powerstat	Time-seri es monitorin g of system power consump tion.	System- wide. Reports on hardware domains via RAPL.	Comman d-line.	sudo powersta t -R 5 12	vmstat-li ke output is ideal for logging power over time and correlatin g with workload changes. Simple and scriptabl e. ⁴⁴	Does not provide per-proc ess attributio n. Focus is on monitorin g, not deep profiling of a single comman d.

AMDuProfCLI	Performance and power profiling specifically for AMD "Zen" processors.	System-wide, per-core.	Command-line.	AMDuProfCLI timechart --event power./my_app	Vendor-supported tool providing detailed, AMD-specific metrics. Part of a larger suite with a GUI. ⁴⁶	Vendor-specific; not applicable to Intel systems. Requires separate installation of the uProf suite.
--------------------	--	------------------------	---------------	---	--	--

Section 5: perf: The Swiss Army Knife for Performance Analysis

The perf tool, sometimes referred to as perf_events, is the official performance analysis utility for the Linux kernel. It is a powerful, low-overhead tool capable of profiling a vast array of hardware and software events, from CPU cycles and cache misses to kernel tracepoints.⁴⁰ Since the release of kernel version 3.14,

perf gained the ability to directly access RAPL energy data, making it a first-line tool for any energy measurement task.⁴⁰

Using perf stat for Energy Measurement

The primary subcommand for gathering aggregate statistics is perf stat. It works by running a specified command and, upon its completion, printing a summary of the events that occurred during its execution.⁴⁸

perf exposes the RAPL domains as kernel Performance Monitoring Unit (PMU) events, which can be listed via perf list | grep power/. The key events are ⁴⁹:

- power/energy-pkg/: Total energy for the entire CPU package(s).
- power/energy-cores/: Energy for the CPU cores only.
- power/energy-ram/: Energy for the DRAM (on supported server platforms).
- power/energy-gpu/: Energy for the integrated GPU.

There are two primary modes of operation for energy measurement:

1. **Per-Command Measurement:** This is the most common use case for analyzing a specific program. The perf stat command is simply prepended to the command you wish to measure.

Bash

```
# Measure the package energy consumed by the 'ls -R /' command  
$ perf stat -e power/energy-pkg/ ls -R / > /dev/null
```

Performance counter stats for 'ls -R /':

```
15.43 Joules power/energy-pkg/
```

```
2.508688169 seconds time elapsed
```

2. **System-Wide Measurement:** To measure the total energy consumption of the entire system over a period, the -a (or --all-cpus) flag is used. This is essential, as RAPL counters are system-wide entities.

Bash

```
# Measure system-wide package and core energy over a 10-second sleep  
$ sudo perf stat -a -e power/energy-pkg/,power/energy-cores/ sleep 10
```

Performance counter stats for 'system wide':

```
85.12 Joules power/energy-pkg/
```

```
21.34 Joules power/energy-cores/
```

```
10.001345891 seconds time elapsed
```

Note that system-wide measurement typically requires root privileges.

Interpreting perf stat Output

The output of perf stat for energy events is straightforward. It provides two key pieces of information:

- The total **energy** consumed during the command's execution, measured in **Joules**.
- The total wall-clock **time** elapsed.

From these two values, one can easily calculate the average power draw in Watts:

Power(W)=Energy(J)/Time(s)

In the first example above, the average power consumption of the system while `ls` was running was $15.43\text{J}/2.509\text{s} \approx 6.15\text{W}$. It is also important to understand the relationship between the domains. The package energy (`energy-pkg`) is a superset of the core and integrated GPU energy, meaning the following relationship generally holds: $\text{energy-pkg} \geq \text{energy-cores} + \text{energy-gpu}$.⁴⁹ The difference accounts for other uncore components like the L3 cache and memory controller.

Section 6: `powertop`: Diagnosing and Tuning System Inefficiencies

While `perf` is excellent for measuring the energy cost of a specific, active task, `powertop` is designed to solve a different problem: finding and fixing sources of energy waste when the system *should* be idle. It is a diagnostic tool developed by Intel to help users identify applications and kernel drivers that are frequently waking up the CPU, preventing it from entering deep, power-saving idle states (known as C-states).⁴²

Interactive Mode

Running `sudo powertop` launches an interactive, text-based user interface with several tabs for analysis:

- **Overview:** This main screen provides a list of the top power-consuming processes and device drivers. It highlights components that are sending the most "wakeups per second" to the CPU. A high number of wakeups from a seemingly idle application is a classic "power bug" that `powertop` is designed to find.⁴³ On a calibrated, battery-powered system, this tab will also show a power estimation in Watts for each item.
- **Idle stats:** This tab displays statistics on how much time the processor's cores are spending in different C-states. For optimal power saving, the goal is to maximize the residency in the deeper C-states (e.g., C6, C7), which represent lower power modes.
- **Frequency stats:** This tab shows the time spent in different P-states (performance states), which correspond to different CPU frequencies.
- **Tunables:** This is `powertop`'s most actionable tab. It presents a list of various system settings that can affect power consumption, labeling each with its current state ("Good" for power-saving, "Bad" for performance-oriented). Users can press Enter on any tunable to toggle its state, allowing them to experiment with the effects of different power-saving features, such as disabling "Wake-on-LAN" or enabling autosuspend for USB devices.⁴²

Reporting and Automation

For non-interactive use, powertop offers powerful reporting and automation features:

- **HTML Reports:** To generate a comprehensive report for offline analysis, use the command `sudo powertop --html=report.html`. This creates a detailed file containing all the information from the interactive tabs, which can be shared for diagnostic purposes.⁴³
- **Auto-Tuning:** The command `sudo powertop --auto-tune` will automatically set all tunable parameters to their "Good" setting without user interaction.⁴² This can be a quick way to apply all recommended power-saving settings, for example, via a systemd service at boot. However, it should be used with some caution, as certain tunables (like aggressive USB autosuspend) might interfere with the functionality of specific hardware.⁵¹

Calibration

For powertop to provide accurate power estimations on the Overview tab (especially on laptops), its power estimation engine must first be calibrated. This is done by running the system on battery power and executing `sudo powertop --calibrate`. This process will run for several minutes, cycling through different workloads, screen brightness levels, and device states to build a power model for the specific hardware.⁴²

Section 7: powerstat: vmstat for Power Consumption

If `perf` is for profiling a single execution and `powertop` is for diagnosing idle-state inefficiencies, `powerstat` is the tool of choice for continuous, time-series monitoring of power consumption. Its design and output are intentionally similar to classic Linux monitoring tools like `vmstat` and `iostat`, making it intuitive for system administrators.⁴⁴ It excels at showing how a system's power draw changes over time in response to varying workloads.

Key Features and Flags

powerstat is invoked with an optional interval and count, just like vmstat. The basic syntax is powerstat [interval][count].⁴⁴

- **Using the RAPL Backend:** By default on older systems, powerstat might rely on reading battery status from ACPI, which can be slow to update and less precise. For all modern Intel-based systems, it is essential to use the -R flag to instruct powerstat to read data directly from the RAPL domains via the powercap interface. This provides much more accurate and high-frequency data.⁵⁵

Bash

```
# Measure power every 5 seconds for a total of 12 samples, using the RAPL interface
```

```
$ sudo powerstat -R 5 12
```

- **Other Useful Flags:** powerstat offers a range of options to customize its output⁴⁴:
 - -H: Displays a histogram of the power measurements at the end of the run, showing the distribution of power consumption.
 - -a: Enables all additional statistics, including CPU C-state residency, temperatures, and CPU frequencies.
 - -c: Outputs the data in a script-friendly CSV format.
 - -D: On systems with RAPL, this will show a detailed breakdown of all available RAPL domains (pkg, core, dram, etc.), not just the total.

Interpreting Output

The standard output of powerstat is a table showing a time-stamped series of measurements. Each row represents one sample, showing statistics like CPU utilization, average frequency, and, most importantly, the power consumption in Watts. At the end of the run, it provides a summary with the average, minimum, maximum, and standard deviation of the power readings, giving a clear statistical overview of the monitoring session.⁴⁵ This format is ideal for capturing the power profile of a long-running application that has distinct phases of operation (e.g., initialization, processing, and finalization).

Section 8: Profiling on AMD Architectures

The tools discussed so far—perf, powertop, and powerstat—primarily leverage the Intel RAPL interface via the kernel's powercap framework. While AMD processors also support power

measurement, the ecosystem and tooling are different. The primary vendor-supported tool for this purpose is AMD uProf.

Introduction to AMD uProf

AMD uProf (short for "μProf" or "MICRO-prof") is a comprehensive performance and power analysis suite for applications running on systems with AMD "Zen"-based processors.⁴⁶ It is a standalone package that must be installed separately and includes both a graphical user interface and a powerful command-line tool,

AMDuProfCLI. It provides access to a wide range of performance counters and metrics unique to AMD's architecture.

Command-Line Interface (AMDuProfCLI)

For scripting and server-side analysis, AMDuProfCLI is the primary interface. While it has many modes for performance analysis (e.g., time-based sampling, cache analysis), the key command for energy measurement is `timechart`.⁴⁶

- **The `timechart` Command:** This command is used to collect system-level characteristics like power, temperature, and frequency over the duration of a program's execution. It samples these metrics at regular intervals and saves the results for later analysis.
- **Example Usage:** To profile the power consumption of a specific application, you can use a command similar to the following:

```
Bash
# Collect power metrics for cores 0-3 at 10ms intervals while running my_app
$ AMDuProfCLI timechart --event core=0-3,power --interval 10
-o./amd_profile_output./my_app
```

This command instructs uProf to monitor the power event for a specific set of cores (core=0-3), sample every 10 milliseconds (--interval 10), and save the results to the specified output directory (-o./amd_profile_output).⁴⁶ The resulting data can then be analyzed to understand the application's power profile.

The key takeaway for practitioners is that while the principles of energy measurement are universal, the specific tools can be vendor-dependent. On Intel systems, a rich ecosystem of tools has been built on top of the standardized powercap kernel interface. On AMD systems, the vendor-provided uProf suite is the most direct and feature-rich solution for accessing

detailed power and performance data.

Part 4: Advanced Methodologies and Modern Environments

Moving beyond the use of basic command-line tools, conducting rigorous and meaningful energy analysis requires a more disciplined, scientific approach. This is particularly true in modern computing environments, where virtualization and containerization add layers of complexity that obscure the direct link between a running application and the underlying hardware. This part explores the methodologies for designing robust experiments and introduces the advanced tools necessary for measuring energy consumption in containerized and cloud-native systems.

Section 9: Designing a Robust Measurement Experiment

The quality of any energy measurement is only as good as the quality of the experiment that produces it. Haphazard measurements on a "noisy" system will yield results that are unreliable and not reproducible. To obtain meaningful data, especially for comparative analysis, a structured experimental design is paramount.

Establishing a Stable Baseline

The first and most critical step in any measurement process is to understand the system's baseline power consumption. Hardware consumes a significant amount of power even when idle, and this baseline must be accounted for.

- **Minimize System Noise:** Before running any tests, the system under test should be brought to a quiescent state. This involves shutting down all unnecessary applications, background services, cron jobs, and daemons that are not essential for the program's execution.²¹ On a server, this might involve stopping services like web servers or databases; on a desktop, it means closing web browsers, email clients, and other user applications. The goal is to minimize random fluctuations in system activity that could interfere with the measurement of the target program.

- **Measure Idle Power:** Once the system is quiet, a baseline measurement of its idle power draw should be taken over a significant period (e.g., several minutes) using a tool like powerstat. This provides the E_{idle} value—the energy the system consumes just by being on. This baseline is essential for the "subtraction" method of per-process attribution discussed later.¹⁷

Defining Reproducible Workloads

For measurements to be comparable, especially when evaluating the impact of a code change, the work performed by the application must be identical across runs.

- **Automate Execution:** Relying on manual user interaction to drive a test is not reproducible. The workload should be defined in an automated script or a dedicated test framework (e.g., a unit test suite). This ensures that the application executes the exact same sequence of operations in the same order every time it is measured.¹
- **Repeat and Average:** A single measurement can be subject to random variations in system state (e.g., kernel scheduling, network jitter). To produce a statistically robust result, the entire measurement process—running the automated workload and recording the energy consumption—should be repeated multiple times (e.g., 5 or 10 times). The final reported energy consumption should be the average of these runs. This helps to smooth out anomalies and increases confidence in the result.¹⁷

The Challenge of Per-Process Energy Attribution

This is one of the most significant challenges in software-based energy measurement. As established earlier, hardware interfaces like Intel RAPL report energy consumption at the level of a physical domain (e.g., the entire CPU package). They have no intrinsic knowledge of which software process or container was responsible for that consumption.²² When multiple processes are running concurrently, the total energy measured by RAPL is a composite of all their activities. Attributing this total back to the individual processes requires an estimation model.

- **Strategy 1: Isolation and Subtraction (Single-Process Approximation):** This is the simplest and most common method, but it is only effective on a system that can be made almost completely idle.
 1. Bring the system to a quiescent state.
 2. Measure the total energy consumed by the system while the target program runs for a duration T . Call this E_{total} .

3. Measure the total energy consumed by the idle system for the same duration T. Call this E_{idle} .
4. The energy attributed to the program is then approximated as: $E_{program} \approx E_{total} - E_{idle}$.¹⁷

While straightforward, this method's accuracy degrades rapidly as the amount of other system activity increases, as it incorrectly attributes all non-idle energy consumption to the single target program.

- **Strategy 2: Proportional Attribution (Multi-Process Estimation):** This is the more sophisticated approach used by modern, advanced monitoring tools. It works by distributing the total energy consumed in a time interval among all active processes in proportion to their consumption of a proxy resource, most commonly CPU time.
 1. In a short time interval (e.g., 1 second), measure the total energy consumed by the CPU package using RAPL. Let's say this is 10 Joules.
 2. In the same interval, use kernel accounting features (e.g., reading from the /proc filesystem or using performance counters) to determine the CPU time consumed by each active process.
 3. For example, Process A used 0.3 CPU-seconds, Process B used 0.1 CPU-seconds, and all other system activity used 0.1 CPU-seconds, for a total of 0.5 CPU-seconds of activity.
 4. The total energy (10 J) is then attributed proportionally:
 - Process A's share: $10J \times (0.3/0.5) = 6J$
 - Process B's share: $10J \times (0.1/0.5) = 2J$
 - System's share: $10J \times (0.1/0.5) = 2J$

This model is the foundation of tools like Scaphandre and Kepler, which apply this logic continuously to provide per-process and per-container energy metrics.³² While still an estimation, it is far more accurate in multi-tenant environments than the simple subtraction method.

Section 10: Energy Measurement in Containerized Environments

The rise of containers and orchestration platforms like Kubernetes has revolutionized software deployment, but it has also exacerbated the challenge of energy measurement. A single physical node may host hundreds of containers belonging to different applications and tenants, making the simple "isolation and subtraction" method completely infeasible. This environment demands tools specifically designed for disaggregation and proportional attribution.

Scaphandre

Scaphandre is a versatile, open-source energy metrology agent designed to be lightweight and adaptable to various environments, including bare-metal servers, virtual machines, and containerized systems.⁶¹

- **Architecture:** Written in the Rust programming language for performance and safety, Scaphandre is built on a modular architecture consisting of two main components⁶¹:
 1. **Sensor:** This component is responsible for collecting the raw power consumption data from the host system. On Linux, it typically uses the `powercap_rapl` sensor, which reads data from the kernel's `powercap sysfs` interface.⁶³
 2. **Exporter:** This component takes the data collected by the sensor and sends it to a specified destination for storage and analysis. This modular design makes Scaphandre highly flexible.
- **Use Cases and Integration:** Scaphandre can measure the energy consumption of bare-metal hosts and, importantly, can attribute that consumption to running QEMU/KVM virtual machines.⁶¹ Its most powerful feature is its ability to provide per-process energy metrics, which can then be mapped to containers. It supports a wide range of exporters, allowing it to integrate seamlessly into modern observability stacks. It can expose metrics in the Prometheus format over an HTTP endpoint, push metrics to a Prometheus Pushgateway or a Riemann server, or simply write them to a JSON file.⁶¹ This native Prometheus support makes it an excellent choice for environments that already use Prometheus and Grafana for monitoring, enabling the creation of detailed energy consumption dashboards.⁶⁴

Kepler (Kubernetes-based Efficient Power Level Exporter)

Kepler is the de facto standard for energy monitoring within Kubernetes environments. It is a Cloud Native Computing Foundation (CNCF) sandbox project specifically designed to measure the energy consumption of pods and containers at scale.⁶⁵

- **Architecture:** Kepler is typically deployed as a Kubernetes DaemonSet, which ensures that an instance of the Kepler agent runs on every node in the cluster.⁶⁷ Its core innovation lies in its use of the extended Berkeley Packet Filter (eBPF), a powerful and highly efficient kernel technology. Kepler uses eBPF to probe kernel tracepoints and CPU performance counters, allowing it to collect detailed resource utilization statistics for every process and container with very low overhead.⁶⁵
- **Power Model:** Kepler's power attribution model is a sophisticated implementation of the proportional attribution strategy.⁶⁰

1. On each node, the Kepler agent reads the total node-level energy consumption from hardware sources, primarily Intel RAPL via powercap.
 2. Simultaneously, it uses its eBPF probes to track the CPU time and other resource usage metrics for every container running on that node.
 3. It then applies a ratio-based model to distribute the total node energy among the containers based on their share of resource consumption.⁶⁶
 4. For cloud environments or virtual machines where direct access to RAPL is not available, Kepler can operate in a model-based mode. It uses a pre-trained machine learning model to *estimate* the node's power consumption based solely on the resource utilization metrics (CPU usage, cache misses, etc.) collected by eBPF.⁶⁷
- **Metrics and Integration:** Kepler exports a rich set of metrics in the Prometheus format, which can be scraped by a Prometheus server and visualized in Grafana. Key metrics include `kepler_container_joules_total`, which provides a cumulative energy count for each container, and `kepler_node_platform_joules_total`, which reports the total energy for the node. These fine-grained metrics allow DevOps teams and developers to track the energy footprint of individual microservices, identify energy-intensive pods, and make data-driven decisions about resource allocation and optimization.⁶⁸

The evolution from basic tools to sophisticated agents like Scaphandre and Kepler reflects a critical shift in the field. As computing environments have grown more abstract and complex, the problem of energy measurement has transformed from simply *reading* a hardware sensor to accurately *attributing* that reading across many layers of virtualization and orchestration. These modern tools provide the necessary mechanisms for disaggregation, making energy measurement a tractable problem even in the most complex cloud-native deployments.

Section 11: Building Custom Solutions with PowerAPI

While off-the-shelf tools like perf, Scaphandre, and Kepler cover a wide range of use cases, there are scenarios—particularly in research and highly specialized industrial settings—where a more flexible and customizable approach is needed. For these situations, PowerAPI provides a powerful framework for building bespoke, software-defined power meters.

Overview

PowerAPI is not a single, monolithic tool but rather a Python-based toolkit or framework designed for the construction of custom energy measurement pipelines.⁷⁰ Developed by the Spirals research group at Inria and the University of Lille, its core philosophy is to provide a set

of modular, composable building blocks that can be assembled "à la carte" to create a power meter tailored to specific requirements.⁷⁰ This allows researchers and engineers to experiment with different data sources, power models, and storage backends without having to build an entire measurement system from scratch.

Architectural Concepts

The architecture of a PowerAPI-based application is centered on a flow of data through a pipeline of distinct, interchangeable components. The two most fundamental components are the Sensor and the Formula.⁷⁴

- **Sensor:** A Sensor is an independent component responsible for collecting raw data (metrics) that are correlated with power consumption. It acts as the primary data source. The Sensor runs on the machine being monitored and queries hardware or OS interfaces to gather this data. PowerAPI provides sensors for various sources, including:
 - Reading Hardware Performance Counters (HPCs) via the `perf_event` interface.
 - Monitoring CPU utilization by reading from the `/proc` filesystem.
 - Reading energy values directly from the `powercap sysfs` interface (for RAPL).The collected data is then pushed to a database or message queue, making it available for the next stage of the pipeline.
- **Formula:** A Formula is the computational heart of the power meter. It is a module that consumes the raw data produced by a Sensor and applies a power model to it to compute an estimation of energy consumption.⁷⁴ This is where the core logic and intelligence of the power meter reside. A Formula can be as simple as a linear regression model based on CPU utilization or as complex as a machine learning model that takes dozens of hardware performance counters as input. PowerAPI allows developers to implement their own custom formulas, enabling research into new and more accurate power models.
- **Actor-Based Model and Pipeline:** PowerAPI is built on an actor-based model, which promotes a modular and extensible architecture. A complete power meter is constructed by connecting these components into a pipeline⁷²:
 1. A **Puller** retrieves data from a source (e.g., a MongoDB database where a Sensor has stored its readings).
 2. The data is passed to a **Formula** actor, which calculates the power estimation.
 3. The resulting power data is then sent to a **Pusher**, which stores the final result in a desired destination (e.g., another database, a CSV file, or a web endpoint).

This flexible, plug-and-play architecture allows a user to easily swap components. For instance, one could test the same Formula with data from two different Sensors to see which provides better input, or send the output of a single measurement pipeline to multiple storage

backends simultaneously. This modularity makes PowerAPI an ideal platform for research and development in the field of software energy measurement.

Part 5: From Measurement to Mitigation

The ultimate goal of measuring energy consumption is not merely to collect data, but to act upon it. Measurement is the essential first step that enables the subsequent, more important goal of mitigation: the creation of more energy-efficient and sustainable software. This final part of the report closes the loop by connecting the "how" of measurement to the "what" of optimization. It outlines actionable green software engineering practices and looks toward the future of this rapidly evolving field.

Section 12: Actionable Green Software Engineering Practices

Green software engineering is the practice of designing, developing, and deploying software in a way that results in fewer greenhouse gas emissions.⁷⁶ This is a holistic approach that considers efficiency at every stage of the software development lifecycle, from the choice of algorithm to the management of cloud infrastructure.⁸ The data gathered from the measurement tools and techniques described in the previous sections provides the feedback necessary to guide these practices.

Energy-Aware Coding and Design

The most direct impact a developer can have on a program's energy footprint is through the code they write and the design decisions they make.

- **Algorithms and Data Structures:** This is the most fundamental level of optimization. The computational complexity of an algorithm has a direct correlation with the number of hardware operations required to execute it. Choosing an algorithm with a lower complexity class (e.g., $O(n \log n)$ instead of $O(n^2)$) is the single most effective way to reduce both execution time and energy consumption for a given task.⁷⁷
- **Programming Language Choice:** The choice of programming language can have a dramatic impact on energy efficiency. Studies have shown that for compute-intensive workloads, compiled languages such as C, C++, and Rust can be orders of magnitude

more energy-efficient than interpreted languages like Python, Perl, or Ruby.⁸ This is because compiled languages translate directly to optimized machine code, minimizing the overhead of runtime interpretation or just-in-time compilation. This decision, however, involves a significant trade-off with developer productivity, and the right choice depends on the specific requirements of the application.

- **Data Handling and Network Efficiency:** Data movement is energetically expensive, whether it is between memory and CPU, between disk and memory, or across a network. Green coding practices focus on minimizing this movement.
 - Reduce the size of data payloads through efficient serialization formats (e.g., Protocol Buffers instead of JSON) and data compression.⁹
 - Utilize caching strategies effectively at all levels (CPU cache, application-level cache, Content Delivery Networks) to avoid re-computing data or re-fetching it from a remote source.⁷⁸
 - Optimize network requests by batching multiple small requests into a single larger one to reduce the overhead of establishing connections and to allow the network hardware to enter low-power states.⁸²

Architectural Patterns and Their Energy Implications

Software architecture and design patterns, while often chosen for reasons of maintainability, flexibility, and scalability, have significant and often overlooked energy implications.

- **The Cost of Abstraction:** Many common object-oriented design patterns, such as the Visitor, Decorator, or Abstract Factory patterns, introduce additional layers of indirection, more object allocations, and increased method calls. While these patterns can improve the structural quality of the code, this added complexity can lead to increased CPU and memory usage, resulting in higher energy consumption.⁸³ For example, studies have shown that removing the Visitor pattern from a C++ application can lead to energy reductions of over 66% in some cases.⁸³
- **Energy-Aware Design Patterns:** In response to these challenges, a new catalog of design patterns specifically aimed at improving energy efficiency is emerging. These patterns provide best-practice solutions for common energy-related problems. Examples include ⁸²:
 - **Race-to-Idle:** Perform work as quickly as possible and then allow the hardware to return to a deep, low-power idle state.
 - **Batch Operations:** Group together multiple small operations to reduce transactional overhead.
 - **Dark UI:** On devices with OLED screens, using a dark user interface can significantly reduce the energy consumed by the display.
- **Microservices vs. Monoliths:** The shift from monolithic architectures to microservices

can have a mixed impact on energy efficiency. On one hand, microservices allow for more granular scaling and better resource allocation, as individual services can be scaled independently, potentially reducing waste from over-provisioned resources.⁸⁶ On the other hand, a microservices architecture dramatically increases network communication between services, and this network overhead has its own significant energy cost.⁹ The overall energy impact depends on a careful balance of these factors.

Optimizing for the Entire Stack

A truly sustainable approach requires looking beyond just the application code to the entire ecosystem in which it operates.

- **Hardware Efficiency and Longevity:** Sustainable software should be designed to run efficiently on existing hardware. This reduces the pressure for constant hardware upgrades, thereby minimizing the significant environmental impact of manufacturing new servers (known as "embodied carbon") and reducing electronic waste.⁸
- **Cloud and DevOps Practices:** The cloud offers powerful levers for sustainability.
 - **Region Selection:** Major cloud providers publish data on the carbon intensity of the energy grids that power their data centers. Choosing to run workloads in regions powered primarily by renewable energy can drastically reduce the carbon footprint of the software.⁸
 - **Dynamic Scaling:** Utilize auto-scaling features to dynamically match the number of running server instances to the current demand. This avoids the common and wasteful practice of maintaining a large fleet of idle servers, which consume power without performing useful work.⁸⁷
 - **Green CI/CD:** Integrate energy measurement and carbon intensity checks directly into Continuous Integration and Continuous Deployment (CI/CD) pipelines. Builds that cause a significant regression in energy efficiency can be automatically flagged or even failed, making energy a first-class quality metric alongside performance and correctness.⁸⁸

Section 13: The Future of Sustainable Computing

The field of sustainable software engineering is rapidly evolving from a set of best practices into a mature engineering discipline. As the urgency of the climate crisis and the economic pressures of rising energy costs intensify, the importance of this field will only continue to

grow. Several key trends are shaping its future.

Emerging Trends

- **AI-Driven Optimization:** While AI workloads are a major driver of increased energy consumption, AI and machine learning are also becoming powerful tools for mitigation. Sophisticated ML models are being developed to predict and optimize energy usage in real-time across entire data centers. These systems can dynamically shift workloads, manage cooling systems, and optimize server utilization to minimize power consumption on a massive scale.⁷⁹
- **Green DevOps and Carbon-Awareness:** The practice of integrating sustainability metrics into the DevOps lifecycle is becoming more formalized. This involves treating energy and carbon emissions as first-class, observable metrics, just like latency, error rates, and cost. Dashboards that display the real-time carbon intensity of a running service will become more common, allowing operations teams to make dynamic, carbon-aware decisions, such as shifting non-critical workloads to times of day when the energy grid is supplied by more renewable sources (a practice known as "demand shaping").⁸⁷
- **Standardized Accounting and Reporting:** To move beyond ad-hoc measurements, the industry is developing formal frameworks and standards for energy and carbon accounting. The Software Carbon Intensity (SCI) specification, developed by the Green Software Foundation and now being adopted into an ISO standard, provides a standardized methodology for calculating and reporting the rate of carbon emissions for a software application.⁸ Such standards will enable meaningful comparisons between different software products and will be essential for regulatory compliance and transparent ESG reporting.¹⁶

Concluding Thoughts

Sustainable software engineering represents a paradigm shift in how we think about the quality and impact of the code we create. It is no longer sufficient for software to be merely correct, performant, and secure; it must also be efficient and responsible in its consumption of resources. The ability to measure is the foundational prerequisite for the ability to improve.⁸

The journey towards sustainability in software is not an extra burden to be added on top of existing engineering practices. Instead, it is a powerful lens that reinforces and gives new urgency to long-standing principles of good software design. The pursuit of energy efficiency

naturally leads to better performance, lower operational costs, and more robust and streamlined architectures. By mastering the tools and methodologies for measuring energy consumption, developers and engineers are not only contributing to a more sustainable planet but are also honing their craft and building higher-quality software for the future. The code written today will shape the digital and physical world of tomorrow, and the principles of sustainable engineering provide the guideposts for ensuring that this future is both innovative and responsible.

Works cited

1. You can not avoid it! — measuring the energy consumption of software systems has become essential - BL Research, accessed September 6, 2025, <https://www.research-bl.com/you-can-not-avoid-it-measuring-the-energy-consumption-of-software-systems-has-become-essential/>
2. Black software — the energy unsustainability of software systems in the 21st century - Oxford Academic, accessed September 6, 2025, <https://academic.oup.com/ooenergy/article/doi/10.1093/ooenergy/oia011/6961587>
3. How Data Center Energy Use Affects Your Bill, accessed September 6, 2025, <https://www.integrityenergy.com/blog/is-data-center-energy-usage-driving-up-my-energy-bills/>
4. Data Center Energy Consumption: Trends and Environmental Impact - Resources, accessed September 6, 2025, <https://info.pivotalglobal.com/blog/data-center-energy-consumption-environmental-impact>
5. AI has high data center energy costs — but there are solutions | MIT Sloan, accessed September 6, 2025, <https://mitsloan.mit.edu/ideas-made-to-matter/ai-has-high-data-center-energy-costs-there-are-solutions>
6. As generative AI asks for more power, data centers seek more reliable, cleaner energy solutions - Deloitte, accessed September 6, 2025, <https://www.deloitte.com/us/en/insights/industry/technology/technology-media-and-telecom-predictions/2025/genai-power-consumption-creates-need-for-more-sustainable-data-centers.html>
7. What the data centre and AI boom could mean for the energy sector – Analysis - IEA, accessed September 6, 2025, <https://www.iea.org/commentaries/what-the-data-centre-and-ai-boom-could-mean-for-the-energy-sector>
8. Green IT: How green software development can reduce your carbon footprint - SIG, accessed September 6, 2025, <https://www.softwareimprovementgroup.com/green-software-development/>
9. Making software and data architectures more sustainable | McKinsey & Company, accessed September 6, 2025, <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/tech-forward/making-software-and-data-architectures-more-sustainable>

10. (PDF) The hardware and software aspects of energy consumption in the mobile development platform - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/publication/263268809_The_hardware_and_software_aspects_of_energy_consumption_in_the_mobile_development_platform
11. Data Center Energy Needs Could Upend Power Grids and Threaten the Climate | Article, accessed September 6, 2025, <https://www.eesi.org/articles/view/data-center-energy-needs-are-upending-power-grids-and-threatening-the-climate>
12. The Environmental Impact of AI and How Software Development Can Help Reduce It, accessed September 6, 2025, <https://www.daydevs.com/post/the-environmental-impact-of-ai-and-how-software-development-can-help-reduce-it>
13. 5 Reasons to Monitor Power Consumption - Fluke Corporation, accessed September 6, 2025, <https://www.fluke.com/en-us/learn/blog/power-quality/5-reasons-power-energy-use>
14. 5 Benefits Of Using an Energy Monitoring Software - Attune IoT, accessed September 6, 2025, <https://www.attuneiot.com/resources/5-benefits-energy-monitoring-software>
15. Green Software Development: Energy Efficient Programming Guide - Curiosum, accessed September 6, 2025, <https://www.curiosum.com/blog/green-coding-software-development-energy-efficient-programming-languages>
16. What are ESG Frameworks? | IBM, accessed September 6, 2025, <https://www.ibm.com/think/topics/esg-frameworks>
17. How to Accurately Measure the Energy Consumption of Application Software, accessed September 6, 2025, <https://greensoftware.foundation/articles/how-to-accurately-measure-the-energy-consumption-of-application-software/>
18. www.reddit.com, accessed September 6, 2025, https://www.reddit.com/r/ElectricalEngineering/comments/kyaeik/can_someone_explain_to_me_what_is_power_and/#:~:text=So%20it%20is%20a%20measure,consumed%20in%20some%20time%20period.
19. Energy Meter vs Power Meter – Key Differences, Types & Use Cases, accessed September 6, 2025, <https://www.electroind.com/energy-meter-vs-power-meter/>
20. Estimating the Energy Footprint of Software Systems: a Primer¹
¹footnote 1This document is distributed with a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 license. byncsa - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2407.11611v1>
21. Tools to Measure Software Energy Consumption from your Computer - Luís Cruz, accessed September 6, 2025, <https://luiscruz.github.io/2021/07/20/measuring-energy.html>
22. An experimental comparison of software-based power meters: focus on CPU and GPU - Pages Professionnelles Individuelles de l'ENS de Lyon, accessed September 6, 2025,

- https://perso.ens-lyon.fr/laurent.lefevre/pdf/CCGRID2023_Ostapenco_Jay_Lefevre.pdf
23. Energy Management Hardware vs. Software: What Does Your Building Need?, accessed September 6, 2025, <https://rhino.energy/blog/energy-management-hardware-vs.-software-what-does-your-building-need>
 24. (PDF) Comparative Study of Hardware and Software Power Measurements in Video Compression - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/publication/378307776_Comparative_Study_of_Hardware_and_Software_Power_Measurements_in_Video_Compression
 25. Analysis of RAPL Energy Prediction Accuracy in a Matrix Multiplication Application on Shared Memory - SEDICI, accessed September 6, 2025, http://sedici.unlp.edu.ar/bitstream/handle/10915/66260/Analysis_of_RAPL_Energy_Prediction_Accuracy_in_a_Matrix_Multiplication_Application_on_Shared_Memory.pdf?sequence=6&isAllowed=y
 26. Initial Validation of DRAM and GPU RAPL Power Measurements, accessed September 6, 2025, https://web.eece.maine.edu/~vweaver/papers/tech_reports/2015_dram_rapl_tr.pdf
 27. arxiv.org, accessed September 6, 2025, <https://arxiv.org/html/2312.12150v1#:~:text=The%20hardware%20power%20metrics%20are,the%20underlying%20hardware%20being%20used.>
 28. Power Measurement Methods for Energy Efficient Applications - MDPI, accessed September 6, 2025, <https://www.mdpi.com/1424-8220/13/6/7786>
 29. powercap/raplcap: RAPL power capping C interface with multiple implementations - GitHub, accessed September 6, 2025, <https://github.com/powercap/raplcap>
 30. A Quantitative Evaluation of the RAPL Power Control System - Full-Time Faculty, accessed September 6, 2025, <https://people.cs.uchicago.edu/~hankhoffmann/FC2015.pdf>
 31. Red Alert for Power Leakage: Exploiting Intel RAPL-Induced Side Channels - of Zhenkai Zhang, accessed September 6, 2025, <https://zhenkai-zhang.github.io/papers/rapl.pdf>
 32. Measuring Energy Consumption using RAPL in x86-64 CPUs - Medium, accessed September 6, 2025, <https://medium.com/@sagarwal3110/measuring-energy-consumption-using-rapl-in-x86-64-cpus-42beb6205f7a>
 33. libpfm_intel_rapl(3) - Linux manual page - man7.org, accessed September 6, 2025, https://man7.org/linux/man-pages/man3/libpfm_intel_rapl.3.html
 34. Strategies to Measure Energy Consumption Using RAPL During Workflow Execution on Commodity Clusters - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2505.09375v2>
 35. Analysis of RAPL Energy Prediction Accuracy in a Matrix Multiplication Application on Shared Memory - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/publication/322705221_Analysis_of_RAPL_Energy_P

- [rediction_Accuracy_in_a_Matrix_Multiplication_Application_on_Shared_Memory](#)
36. Power Capping Framework - The Linux Kernel documentation, accessed September 6, 2025, <https://docs.kernel.org/power/powercap/powercap.html>
 37. powercap.txt - The Linux Kernel Archives, accessed September 6, 2025, <https://www.kernel.org/doc/Documentation/power/powercap/powercap.txt>
 38. Documentation/power/powercap/powercap.txt - kernel/msm - Git at Google - Android GoogleSource, accessed September 6, 2025, https://android.googlesource.com/kernel/msm/+android-7.1.0_r0.2/Documentation/power/powercap/powercap.txt
 39. Power Capping Linux, accessed September 6, 2025, https://events.static.linuxfound.org/sites/events/files/slides/LinuxConPowerCapping_jpan.pdf
 40. perf (Linux) - Wikipedia, accessed September 6, 2025, [https://en.wikipedia.org/wiki/Perf_\(Linux\)](https://en.wikipedia.org/wiki/Perf_(Linux))
 41. Linux perf Examples - Brendan Gregg, accessed September 6, 2025, <https://www.brendangregg.com/perf.html>
 42. Powertop - ArchWiki, accessed September 6, 2025, <https://wiki.archlinux.org/title/Powertop>
 43. 2.2. PowerTOP | Power Management Guide | Red Hat Enterprise Linux | 7, accessed September 6, 2025, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/power_management_guide/powertop
 44. powerstat man | Linux Command Library, accessed September 6, 2025, <https://linuxcommandlibrary.com/man/powerstat>
 45. ColinIanKing/powerstat: Powerstat measures the power consumption of a machine using the battery stats or the Intel RAPL interface. The output is like vmstat but also shows power consumption statistics. At the end of a run, powerstat will calculate the average, standard deviation and min/max of the gathered data. - GitHub, accessed September 6, 2025, <https://github.com/ColinIanKing/powerstat>
 46. AMD uProf - SURF Service Desk, accessed September 6, 2025, <https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/74227822/AMD+uProf>
 47. AMD μ Prof, accessed September 6, 2025, <https://www.amd.com/en/developer/uprof.html>
 48. perf-stat(1) - Linux manual page - man7.org, accessed September 6, 2025, <https://man7.org/linux/man-pages/man1/perf-stat.1.html>
 49. Perf — Firefox Source Docs documentation - Mozilla, accessed September 6, 2025, <https://firefox-source-docs.mozilla.org/performance/perf.html>
 50. Perf-icient Measuring of Laptop Energy Consumption and CO₂e | by Gabi - Medium, accessed September 6, 2025, <https://medium.com/@gabibeyer/perf-icient-measuring-of-laptop-energy-consumption-and-co2e-0a36d3a6655a>
 51. How to analysis the output of powertop to increase battery lifetime - Ask Ubuntu, accessed September 6, 2025, <https://askubuntu.com/questions/170667/how-to-analysis-the-output-of-powertop>

[op-to-increase-battery-lifetime](#)

52. Powertop tunables: what does it do? - Unix & Linux Stack Exchange, accessed September 6, 2025, <https://unix.stackexchange.com/questions/18279/powertop-tunables-what-does-it-do>
53. The Linux PowerTOP tool -- please post patches to the mailing list instead of using github pull requests, accessed September 6, 2025, <https://github.com/fenrus75/powertop>
54. powertop - a power consumption and power management diagnosis tool. - Ubuntu Manpage, accessed September 6, 2025, <https://manpages.ubuntu.com/manpages/jammy/man8/powertop.8.html>
55. powerstat - a tool to measure power consumption - Ubuntu Manpage, accessed September 6, 2025, <https://manpages.ubuntu.com/manpages/noble/man8/powerstat.8.html>
56. AMD uProf - ARCHER2 User Documentation, accessed September 6, 2025, <https://docs.archer2.ac.uk/data-tools/amd-uprof/>
57. What is the best way to observe energy usage of a program on Linux? - Ask Ubuntu, accessed September 6, 2025, <https://askubuntu.com/questions/1241160/what-is-the-best-way-to-observe-energy-usage-of-a-program-on-linux>
58. arxiv.org, accessed September 6, 2025, <https://arxiv.org/html/2407.11611v2#:~:text=In%20the%20same%20way%20that,the%20system%20more%20energy%20efficient.>
59. Quantifying the Power Consumption of Processes on Linux using Intel RAPL, accessed September 6, 2025, <https://vbn.aau.dk/ws/files/537435089/report.pdf>
60. Kepler, accessed September 6, 2025, <https://sustainable-computing.io/>
61. hubblo-org/scaphandre: Energy consumption metrology agent. Let "scaph" dive and bring back the metrics that will help you make your systems and applications more sustainable ! - GitHub, accessed September 6, 2025, <https://github.com/hubblo-org/scaphandre>
62. A measurement-based approach to analyze the power consumption of the softwarized 5G core - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2502.13879v1>
63. Demoing Scaphandre - Wondering Chimp, accessed September 6, 2025, <https://www.wonderingchimp.com/demoing-scaphandre/>
64. Kubernetes power usage using Scaphandre | Grafana Labs, accessed September 6, 2025, <https://grafana.com/grafana/dashboards/13845-kubernetes-power-usage-using-scaphandre/>
65. Kepler | CNCF, accessed September 6, 2025, <https://www.cncf.io/projects/kepler/>
66. Container-level Energy Observability in Kubernetes Clusters - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2504.10702v1>
67. Nurturing Sustainability: Raising Power Consumption Awareness | kubernetes @ CERN, accessed September 6, 2025, <https://kubernetes.web.cern.ch/blog/2023/09/05/nurturing-sustainability-raising->

- [power-consumption-awareness/](#)
68. Monitoring environmental impact of Kubernetes clusters | by Vasyl Shponarskyi | Medium, accessed September 6, 2025, <https://medium.com/@vasyl.shponarskyi/monitoring-environmental-impact-of-kubernetes-clusters-8c81f72ac55f>
 69. Monitoring Container Power Consumption with Kepler, accessed September 6, 2025, <https://sustainable-computing.io/archive/design/metrics/>
 70. powerapi-ng/powerapi: PowerAPI is a Python framework for building software-defined power meters. - GitHub, accessed September 6, 2025, <https://github.com/powerapi-ng/powerapi>
 71. PowerAPI - GitHub, accessed September 6, 2025, <https://github.com/powerapi-ng>
 72. PowerAPI, accessed September 6, 2025, <https://powerapi.org/>
 73. PowerAPI: A Software Library to Monitor the Energy Consumed at the Process-Level, accessed September 6, 2025, <https://ercim-news.ercim.eu/en92/special/powerapi-a-software-library-to-monitor-the-energy-consumed-at-the-process-level>
 74. Overview - PowerAPI, accessed September 6, 2025, <https://powerapi.org/reference/overview/>
 75. PowerAPI Reference Architecture. | Download Scientific Diagram - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/figure/PowerAPI-Reference-Architecture_fig2_254463770
 76. Green Software Foundation | GSF, accessed September 6, 2025, <https://greensoftware.foundation/>
 77. Good Practices for Sustainable Software Development - BioSistemika, accessed September 6, 2025, <https://biosistemika.com/blog/good-practices-for-sustainable-software-development/>
 78. Best Practices for Sustainable Software Development: Green Efficiency Principles - LaSoft, accessed September 6, 2025, <https://lasoft.org/blog/best-practices-for-sustainable-software-development/>
 79. Best Practices of Sustainable Software Development - Green.org, accessed September 6, 2025, <https://green.org/2024/05/24/best-practices-of-sustainable-software-development/>
 80. Sustainable Software Development: Coding for a Greener Future - Softensity, accessed September 6, 2025, <https://www.softensity.com/blog/sustainable-software-development-coding-for-a-greener-future/>
 81. The Whats and Whys of Green Software Development - Avenga, accessed September 6, 2025, <https://www.avenga.com/magazine/explore-green-software-development/>
 82. Design patterns for energy-efficient Web applications | Chuniverseit, accessed September 6, 2025, <https://chuniverseit.nl/papers/energy-design-patterns>

83. Energy-Aware Pattern Framework: The Energy-Efficiency Challenge for Embedded Systems from a Software Design Perspective | Request PDF - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/publication/349631728_Energy-Aware_Pattern_Framework_The_Energy-Efficiency_Challenge_for_Embedded_Systems_from_a_Software_Design_Perspective
84. On the Energy Consumption of Design Patterns, accessed September 6, 2025, <https://dl.gi.de/bitstreams/25b66baf-b644-423a-b66a-3efd24659399/download>
85. Energy Consumption Analysis of Design Patterns - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/publication/27378122_Energy_Consumption_Analysis_of_Design_Patterns
86. 7 Ways to Practice Green Coding in Your Next Projects - IEEE Computer Society, accessed September 6, 2025, <https://www.computer.org/publications/tech-news/trends/implementing-green-coding-practices/>
87. Sustainable Software Practices in 2025 - Instandart, accessed September 6, 2025, <https://instandart.com/blog/bespoke-software-development/sustainable-software-practices-in-2025/>
88. Green Software Development Best Practices Enhanced by PixelFree Studio's Tools, accessed September 6, 2025, <https://blog.pixelfreestudio.com/green-software-development-best-practices-enhanced-by-pixelfree-studios-tools/>
89. Protik49/12-Open-Source-Tools-for-Monitoring-Software-Energy-Consumption - GitHub, accessed September 6, 2025, <https://github.com/Protik49/12-Open-Source-Tools-for-Monitoring-Software-Energy-Consumption>
90. Designing Algorithms for Battery Optimization: Powering the Future of Mobile Devices, accessed September 6, 2025, <https://algotcademy.com/blog/designing-algorithms-for-battery-optimization-powering-the-future-of-mobile-devices/>
91. The environmental impact of computing: problems and possible solutions - PVcase, accessed September 6, 2025, <https://pvcase.com/blog/the-environmental-impact-of-computing-problems-and-possible-solutions>
92. Why Sustainability in Software Engineering Matters | Leobit, accessed September 6, 2025, <https://leobit.com/blog/why-sustainable-software-engineering-matters/>
93. 7 ESG Frameworks Explained from a Carbon Accounting Lens - Atrius, accessed September 6, 2025, <https://atrius.com/esg-frameworks-for-carbon-accounting/>